Tidtaking for spillkveld i Blazor

Klokke som teller live og fortsetter å gå selv om du bytter side

I stedet for en `Stopwatch`, som bare lever så lenge komponenten gjør, lagres *tidspunktet* spillet ble startet. Da kan tiden regnes ut uansett hvilken side du er på. Siden starttidspunktet lagres sammen med spillene i nettleseren, overlever det også en oppdatering av siden.

1. BoardGame: legg til én egenskap

```
public DateTime? StartedAt { get; set; }
```

`null` betyr at spillet ikke går. Ellers står tidspunktet det ble startet.

2. GameNightState: legg til tre metoder

```
public async Task StartAsync(int gameId)
{
    var game = GetGameById(gameId);
    if (game is null || game.StartedAt is not null) return; // går allerede

    game.StartedAt = DateTime.UtcNow;
    await SaveAsync();
    OnChange?.Invoke();
}

public async Task StopAsync(int gameId)
{
    var game = GetGameById(gameId);
    if (game?.StartedAt is null) return; // går ikke

    game.PlayTime += DateTime.UtcNow - game.StartedAt.Value;
    game.StartedAt = null;
    await SaveAsync();
    OnChange?.Invoke();
}

public TimeSpan GetLiveTime(BoardGame game) =>
    game.StartedAt is null
        ? game.PlayTime
        : game.PlayTime + (DateTime.UtcNow - game.StartedAt.Value);
```

`SetPlayTimeAsync` kan slettes, den trengs ikke lenger.

Hvis totaltiden også skal ta med spill som går nå:

```
public TimeSpan TotalPlayTime =>
    games.Aggregate(TimeSpan.Zero, (sum, g) => sum + GetLiveTime(g));
```

3. StartStopGame: hele komponenten

```
@inject GameNightState GameNight
@implements IDisposable

<p>@LiveTime.ToString(@"hh\:mm\:ss")</p>

<button class="btn btn-outline-danger"
        @onclick="() => StartGame(GameId)">Start spill</button>
<button class="btn btn-primary"
        @onclick="() => EndGame(GameId)">Slutt spill</button>

@code {
    [Parameter] public int GameId { get; set; }
    [Parameter] public EventCallback OnGameEnded { get; set; }

    private PeriodicTimer? timer;

    private TimeSpan LiveTime
    {
        get
        {
            var game = GameNight.GetGameById(GameId);
            return game is null ? TimeSpan.Zero : GameNight.GetLiveTime(game);
        }
    }

    protected override void OnInitialized()
    {
        _ = RunTimerAsync();
    }

    private async Task RunTimerAsync()
    {
        timer = new PeriodicTimer(TimeSpan.FromSeconds(1));

        while (await timer.WaitForNextTickAsync())
        {
            await InvokeAsync(StateHasChanged);
        }
    }

    public async Task StartGame(int id)
    {
        await GameNight.StartAsync(id);
    }

    public async Task EndGame(int id)
    {
        await GameNight.StopAsync(id);
        await OnGameEnded.InvokeAsync();
    }

    public void Dispose() => timer?.Dispose();
}
```

Bruk i forelderen

```
@foreach (var game in GameNight.Games)
{
    <p>@game.Name</p>
    <StartStopGame GameId="game.Id" OnGameEnded="StateHasChanged" />
}

<p>Total spilletid: @GameNight.TotalPlayTime.ToString(@"hh\:mm\:ss")</p>
```

Hvordan det fungerer

- **Ingen stoppeklokke.** Tiden regnes ut som «lagret tid + (nå - starttidspunkt)».
- **StartAsync** lagrer når spillet startet. **StopAsync** legger den nye tiden til `PlayTime` og nullstiller starttidspunktet, så Start igjen fortsetter fra lagret tid.
- **Timeren starter når komponenten vises** (`OnInitialized`), ikke ved Start. Kommer du tilbake til siden mens et spill går, teller klokka videre med riktig tid.
- **WaitForNextTickAsync()** venter ett sekund om gangen uten å blokkere siden. Den returnerer `false` når timeren er disposed, og da avsluttes løkka.
- **InvokeAsync(StateHasChanged)** ber Blazor tegne komponenten på nytt på en trygg måte fra en løkke som kjører ved siden av knappeklikkene.
- **Dispose** stopper bare oppdateringen av skjermen når du forlater siden. Selve spillet fortsetter å «gå», fordi starttidspunktet ligger lagret på spillet.
- **DateTime.UtcNow** brukes i stedet for `DateTime.Now`, så sommertid ikke kan få tiden til å hoppe.

Visning av tid

```
@time.ToString(@"mm\:ss")                      // 04:37
@time.ToString(@"hh\:mm\:ss")                  // 01:04:37
@((int)time.TotalMinutes) min @time.Seconds sek  // 64 min 37 sek
```

Bruk `TotalMinutes`, ikke `Minutes`, når timene skal regnes med: 64 minutter gir `Minutes` = 4.